RomM-DS64

RETRO GAME MANAGER & DOCKER DEPLOYMENT



MANAGE YOUR ROMS SEAMLESSLY

ROMM OFFICIAL BUILD

RomM (ROM Manager)

Guía Técnica Paso a Paso: Despliegue, Preservación Digital y Gestión de la Biblioteca

---

Proyecto: RomM-DS64 (Orquestación en Contenedores)
Autor: Miguel Ángel Lastra
Software Base: RomM (rommapp/romm), MariaDB 10.11, qBittorrent, Minerva Archive
Fecha de Ejecución: Octubre de 2026

Índice de contenidos

1. Requisitos del Sistema y Marco Legal de Copyright — Pág. 2
2. Fase 1: Despliegue de la Infraestructura con Docker — Pág. 3
3. Fase 2: Asistente de Configuración Inicial (Setup) — Pág. 4
4. Fase 2 (Cont.): Fuentes de Metadatos y Autenticación — Pág. 5
5. Fase 3: Adquisición del Cliente qBittorrent — Pág. 6
6. Fase 4: Repositorios de Preservación en Minerva Archive — Pág. 7
7. Fase 4 (Cont.): Descarga y Verificación con qBittorrent — Pág. 8
8. Fase 5: Carga Masiva y Subida de ROMs a RomM — Pág. 9
9. Fase 5 (Cont.): Monitorización de la Subida por Lotes — Pág. 10
10. Fase 6: Catálogo de la Biblioteca y Emulación Web — Pág. 11
11. Fase 7: Mantenimiento y Buenas Prácticas — Pág. 12

1. Requisitos del Sistema y Marco Legal de Copyright

Antes de comenzar con la instalación y configuración de **RomM (ROM Manager)**, es fundamental definir los requerimientos técnicos indispensables y las consideraciones éticas y legales asociadas a la gestión de copias de seguridad de software interactivo.

Requisitos Técnicos del Entorno:

- **Hardware Anfitrión:** Procesador con arquitectura de 64 bits (x86_64 o ARM64), mínimo 4 GB de memoria RAM (recomendado 8 GB para transcodificación y emulación fluida) y almacenamiento suficiente para albergar la biblioteca (SSD o disco secundario de alta capacidad).
- **Motor de Contenedorización:** Docker Engine versión 20.10 o superior, acompañado de Docker Compose v2.x debidamente configurado en el sistema operativo anfitrión (Windows 11 con WSL2 o distribuciones Linux).
- **Cliente BitTorrent:** Software cliente libre y seguro, específicamente `qBittorrent`, para la descarga descentralizada y validación por hash de colecciones de preservación.
- **Navegador Web Moderno:** Google Chrome, Mozilla Firefox o Microsoft Edge con aceleración por hardware y compatibilidad con WebAssembly/WebGL para permitir la ejecución de emuladores web integrados.
- **Conectividad de Red:** Conexión a Internet de banda ancha para la descarga de imágenes Docker, catálogos No-Intro y consumo de las APIs de metadatos (IGDB/Twitch).

Marco Legal, Propiedad Intelectual y Preservación Digital:

Los videojuegos, su código binario, obras artísticas y bandas sonoras están amparados por las leyes internacionales de propiedad intelectual y derechos de autor (*copyright*). El uso de emuladores y plataformas de gestión autoalojadas como RomM está reconocido como legal siempre que se ajuste a los principios de:

- **Copias de Seguridad Personales (Backups):** El usuario debe poseer legalmente el soporte físico original (cartucho, disco óptico o licencia comercial legítima) del videojuego que almacena y gestiona en su servidor personal.
- **Preservación del Patrimonio Digital:** Proyectos como *No-Intro* y repositorios de preservación histórica como *Minerva Archive* tienen por objetivo catalogar, auditar mediante sumas de verificación criptográfica (hashes CRC32, MD5, SHA-1) y salvaguardar el software frente a la degradación física inevitable de los soportes magnéticos y chips ROM originales.
- **Prohibición Expresa de Comercialización:** Queda estrictamente desaconsejada cualquier distribución con fines lucrativos o puesta a disposición pública no autorizada de material sujeto a derechos vigentes.

2. Fase 1: Despliegue de la Infraestructura con Docker

Una vez preparado el archivo declarativo `docker-compose.yml` y el archivo de variables seguras `.env`, se inicia el despliegue del stack de servicios en la consola de comandos de PowerShell.

Orquestación de los Contenedores (RomM y MariaDB):

En el directorio raíz del proyecto (`RomM-DS64`), se ejecuta la instrucción de inicio en segundo plano:

Terminal PowerShell

```
docker compose up -d
```

Al procesar la orden, Docker Compose realiza automáticamente la secuencia completa de despliegue:

- Descarga las imágenes oficiales requeridas: `rommapp/romm:latest` para la aplicación y `mariadb:10.11` para el gestor de base de datos relacional.
- Inicializa los volúmenes persistentes de Docker: `romm_resources`, `romm_redis_data` y `romm_db_data`.
- Crea la red virtual privada en modo bridge (`romm-ds64_default`) para la interconexión segura entre la aplicación y el motor de base de datos.
- Inicia en orden dependiente el contenedor `romm-db` y posteriormente el contenedor principal `romm`, exponiendo el puerto `8080` en el host anfitrión.

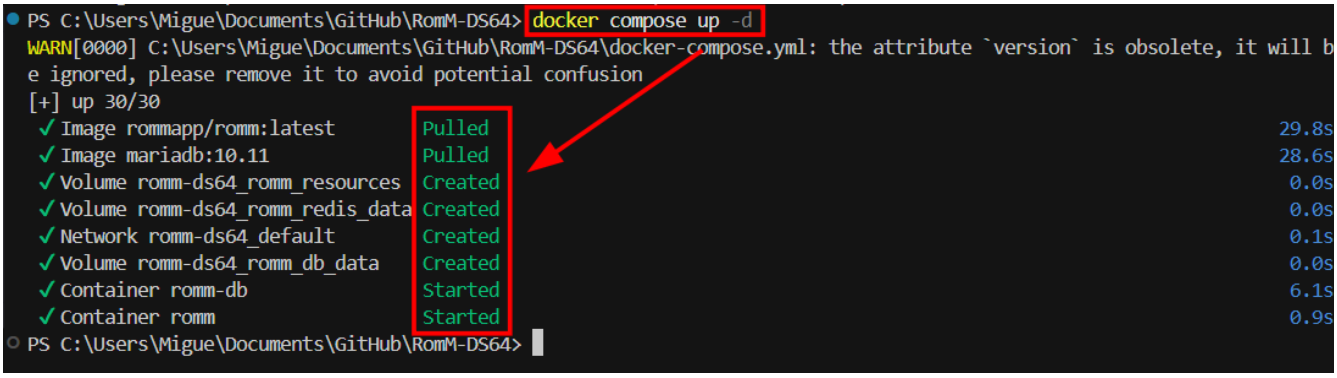


Figura 1. Ejecución de docker compose up -d en PowerShell, descarga de imágenes y levantamiento de servicios.

3. Fase 2: Asistente de Configuración Inicial (Setup)

Al acceder por primera vez a la interfaz web a través del puerto configurado (`http://localhost:8080/setup`), RomM presenta un asistente de bienvenida estructurado en tres pasos clave para inicializar la instancia.

Paso 1 de 3: Configurar Estructura de Biblioteca y Plataformas:

El sistema permite definir la estructura de carpetas de almacenamiento (`roms/{platform}/{game}`) y seleccionar qué consolas formarán parte de la colección. En este caso práctico, se selecciona la plataforma **Nintendo 64** (identificador `n64`) dentro del catálogo de plataformas disponibles, procediendo a la creación de su carpeta dedicada mediante el botón **Siguiente**.

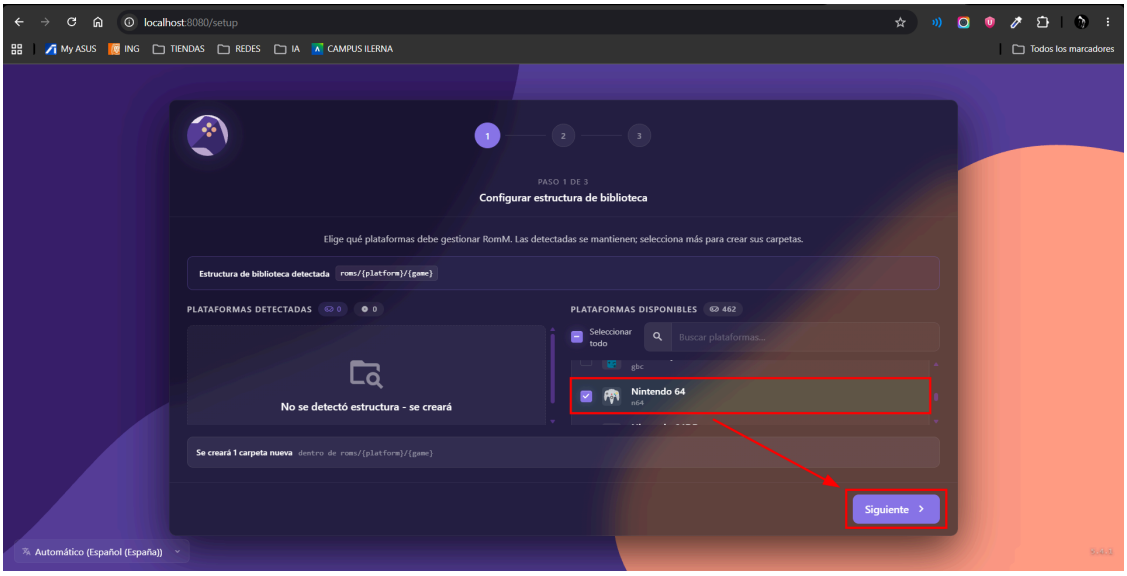


Figura 2. Paso 1 de 3: Selección de la plataforma Nintendo 64 (n64) en el asistente de biblioteca.

Paso 2 de 3: Creación de la Cuenta Administradora:

A continuación, el asistente solicita las credenciales del usuario propietario y administrador principal del servidor. Se definen el nombre de usuario (`Lastra`), la dirección de correo electrónico vinculada y una contraseña robusta para gestionar la biblioteca y los permisos de acceso.

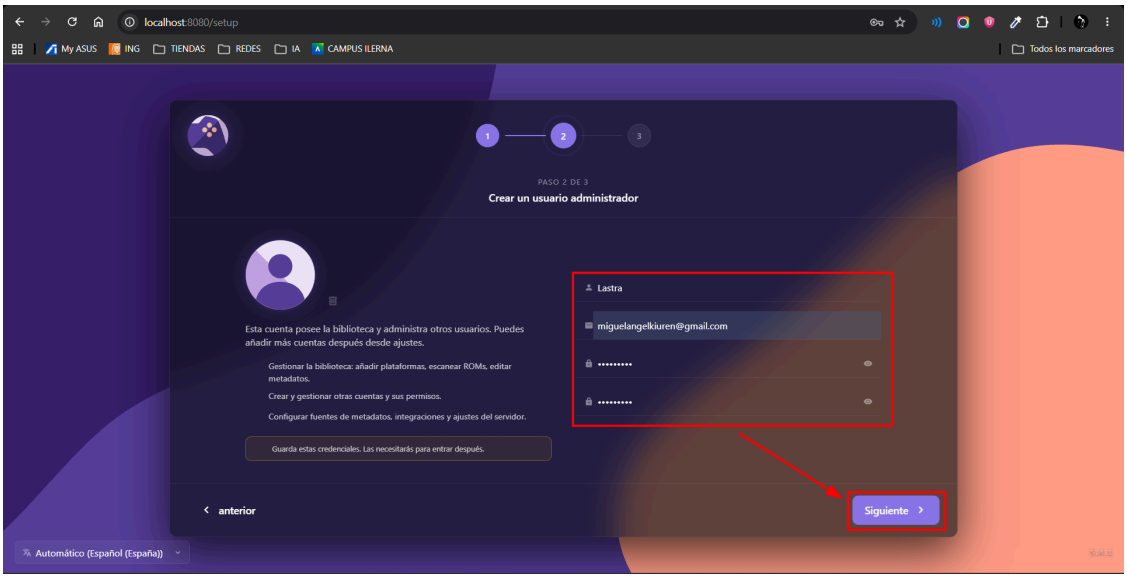


Figura 3. Paso 2 de 3: Registro de la cuenta de administrador principal en RomM.

4. Fase 2 (Cont.): Fuentes de Metadatos y Autenticación

La culminación del asistente técnico asegura que el entorno disponga de los servicios auxiliares adecuados y permite verificar el acceso seguro a la plataforma.

Paso 3 de 3: Comprobación de Proveedores de Metadatos:

En el último paso del asistente, RomM analiza la disponibilidad de los proveedores de metadatos e índices comunitarios (como *Hasheous* y *PlayMatch* para identificación por hash criptográfico, además de la API de IGDB). Una vez comprobadas las fuentes deseadas, se pulsa sobre **Finalizar** para consolidar los esquemas en la base de

datos MariaDB.

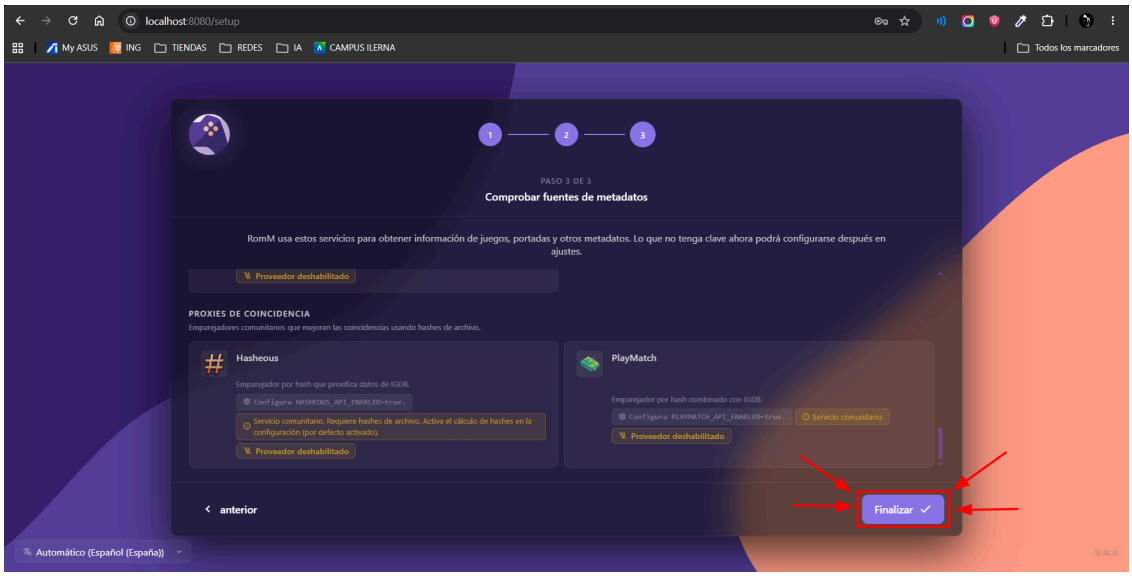


Figura 4. Paso 3 de 3: Verificación de proxies de coincidencia y cierre del asistente inicial.

Autenticación y Acceso al Panel de Control:

Con la inicialización completada, el servidor redirige a la pantalla de autenticación central (`http://localhost:8080/login`). Se introducen el usuario recién creado (`Lastra`) y la contraseña establecida para validar la sesión y acceder al panel de administración.

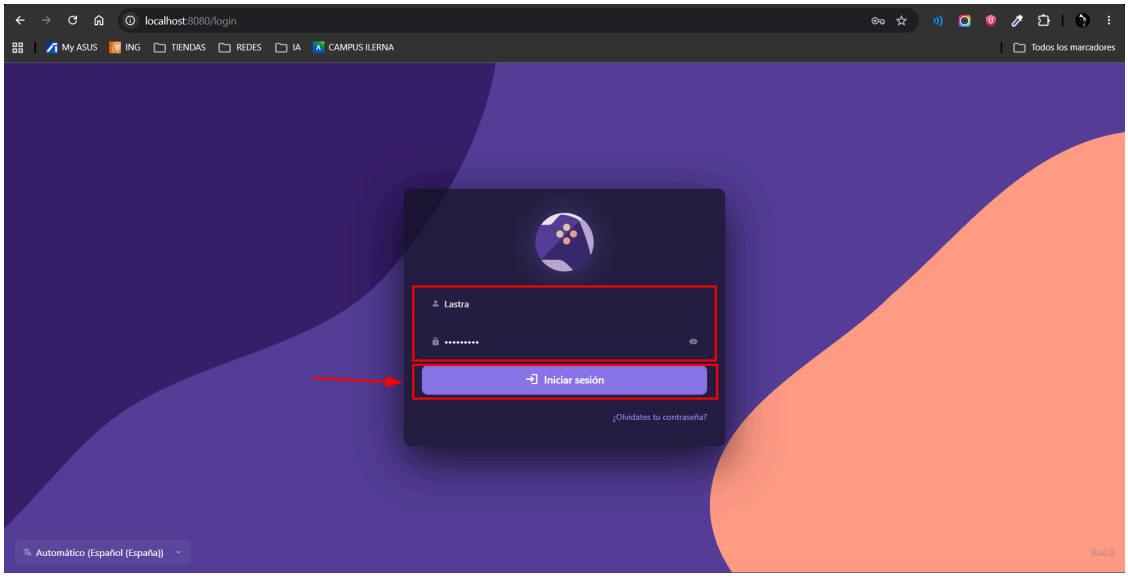


Figura 5. Pantalla de inicio de sesión de RomM para autenticar la cuenta de administración.

5. Fase 3: Adquisición del Cliente qBittorrent

Para gestionar la transferencia de grandes volúmenes de datos y colecciones completas preservadas sin sobrecargar servidores web tradicionales, el protocolo BitTorrent representa el estándar por excelencia de distribución peer-to-peer (P2P).

Descarga del Software Oficial:

Se accede al portal oficial del proyecto en `https://www.qbittorrent.org/download`. qBittorrent es una alternativa de código abierto, libre de software publicitario o rastreadores, desarrollada en C++ con la biblioteca *libtorrent-rasterbar*.

Se selecciona el instalador oficial para entornos de 64 bits (en este caso, **Download qBittorrent v5.3.0** para Windows 10+), procediendo a su instalación estándar en el equipo anfitrión.

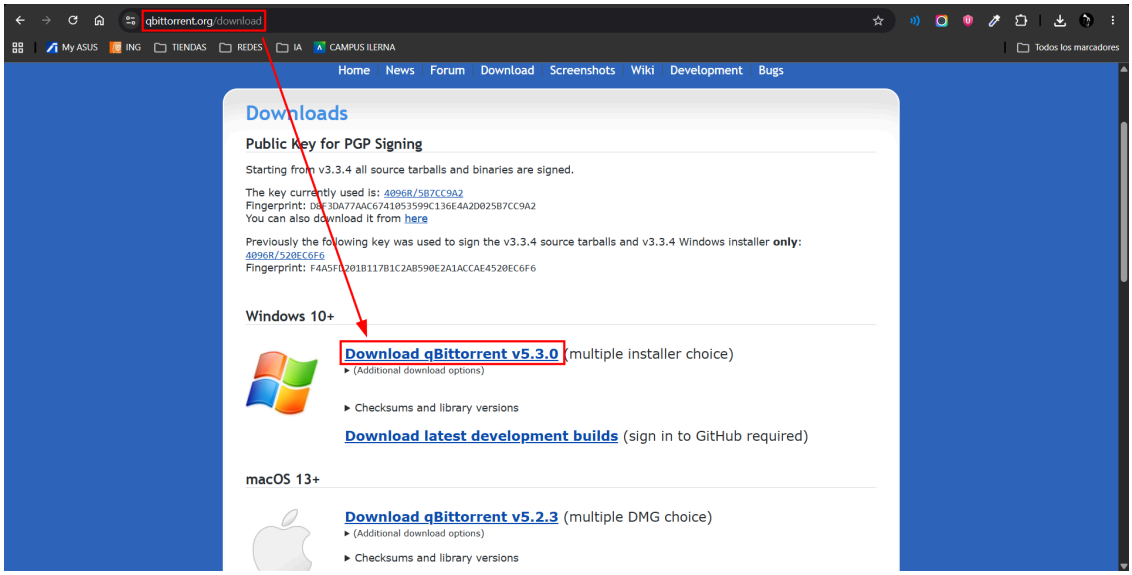


Figura 6. Portal de descargas de qBittorrent seleccionando la versión v5.3.0 para Windows de 64 bits.

Ventajas del Uso de qBittorrent en Preservación:

- **Verificación de Integridad Pieza a Pieza:** Cada bloque transferido se valida contra el hash criptográfico del archivo torrent, asegurando que ninguna ROM sufra corrupción durante la descarga.
- **Selección Granular de Archivos:** Permite inspeccionar el árbol interno del torrent antes de descargarlo, seleccionando exclusivamente los títulos o revisiones específicas deseadas.
- **Control de Ancho de Banda:** Posibilita fijar límites de subida y bajada para no comprometer el tráfico general de la red local.

6. Fase 4: Repositorios de Preservación en Minerva Archive

Para garantizar que los volcados digitales correspondan fielmente a las copias originales de los cartuchos de nuestra propiedad, se utilizan repositorios estandarizados bajo la iniciativa **No-Intro**.

Búsqueda y Localización en Minerva Archive:

Se navega hacia la plataforma de preservación histórica en la dirección `https://minerva-archive.org/browse/No-Intro/`. Este directorio organiza colecciones catalogadas según estándares estrictos de preservación:

- **Catálogos No-Intro:** Conjuntos de datos limpios de copias piratas, malas lecturas (*bad dumps*) o modificaciones no oficiales, conservando únicamente las imágenes binarias puras validadas con sumas hash estándar.
- **Filtrado por Plataforma:** En el buscador integrado se introduce el término `nintendo 64` para localizar los conjuntos específicos del sistema.

- **Selección del Repositorio BigEndian:** Se identifica el paquete `Nintendo - Nintendo 64 (BigEndian)/` (con un tamaño total consolidado de aproximadamente 14.68 GB) y se hace clic en el enlace Magnet correspondiente para asociarlo al cliente de descargas.

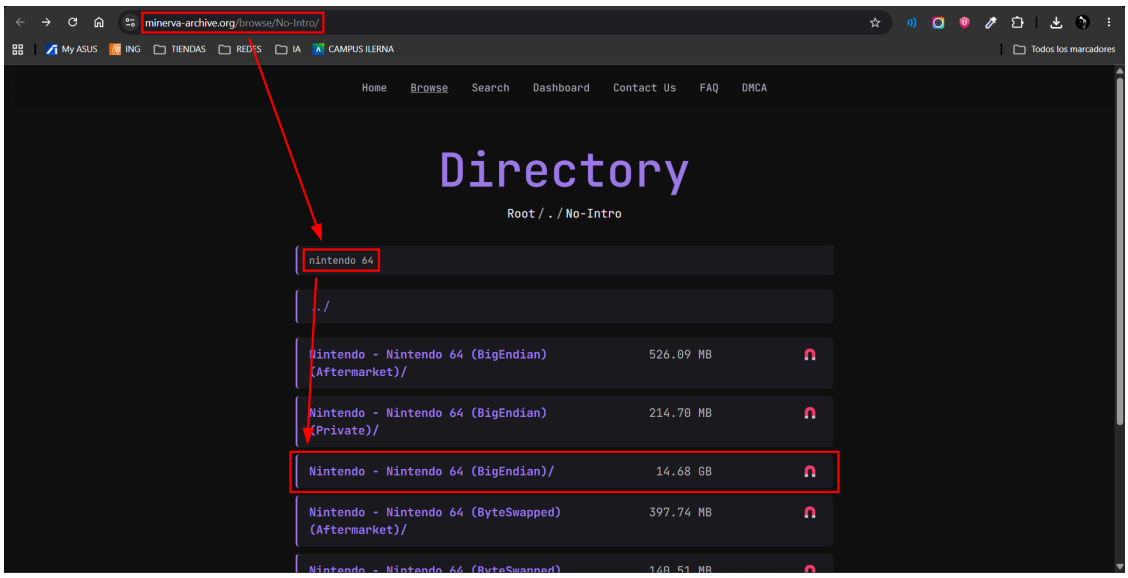


Figura 7. Repositorio Minerva Archive No-Intro: Búsqueda del catálogo de Nintendo 64 (BigEndian).

7. Fase 4 (Cont.): Descarga y Verificación con qBittorrent

Al hacer clic en el enlace Magnet o abrir el archivo torrent en el navegador, el sistema operativo transfiere el enlace automáticamente a **qBittorrent**, abriendo la ventana de parametrización de la transferencia.

Asignación de Directorio y Comprobación de Archivos:

En el cuadro de diálogo de qBittorrent (*Minerva_Myrient*), se configura la recepción de los metadatos y se especifica la ruta de almacenamiento local:

- **Ruta de Destino:** Se selecciona la carpeta `C:\Games` en el campo *Guardar en*, asegurando que el disco cuente con espacio suficiente (48.70 GiB libres en la unidad).
- **Árbol de Contenidos:** Se despliega la jerarquía de ficheros comprimidos (`.zip`) individuales para verificar que contengan los títulos correspondientes a las licencias que poseemos.
- **Inicio de Transferencia:** Se pulsa el botón **Aceptar** para iniciar la descarga secuencial y verificación criptográfica de cada fragmento.

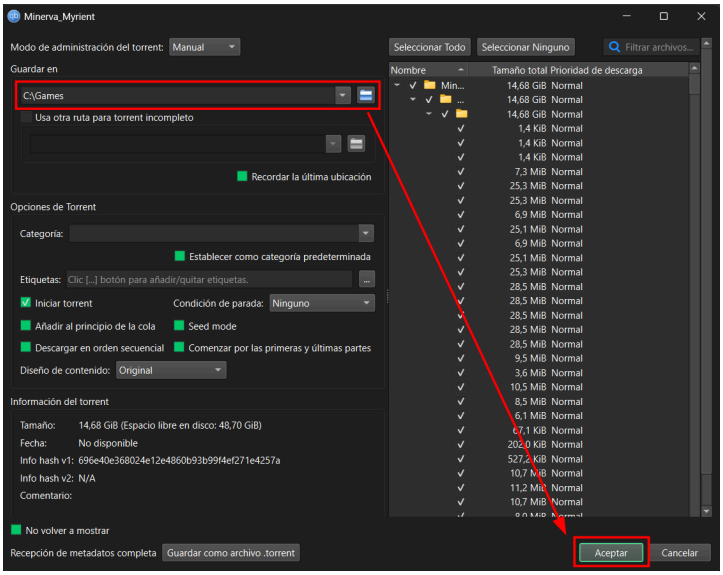


Figura 8. Diálogo de qBittorrent configurando el destino en C:\Games y validando la lista de ROMs.

8. Fase 5: Carga Masiva y Subida de ROMs a RomM

Una vez finalizada la descarga y verificados los archivos en el equipo anfitrión, se procede a alimentarlos en el servidor central de RomM a través de su interfaz de transferencia integrada.

Módulo de Carga Web (Subir ROMs):

En el panel principal de RomM, dentro de la barra de navegación lateral izquierda en la categoría *Biblioteca*, se accede a la opción **Subir ROMs** (`http://localhost:8080/upload`). La plataforma presenta un área de trabajo interactiva basada en arrastrar y soltar (*Drag & Drop*).

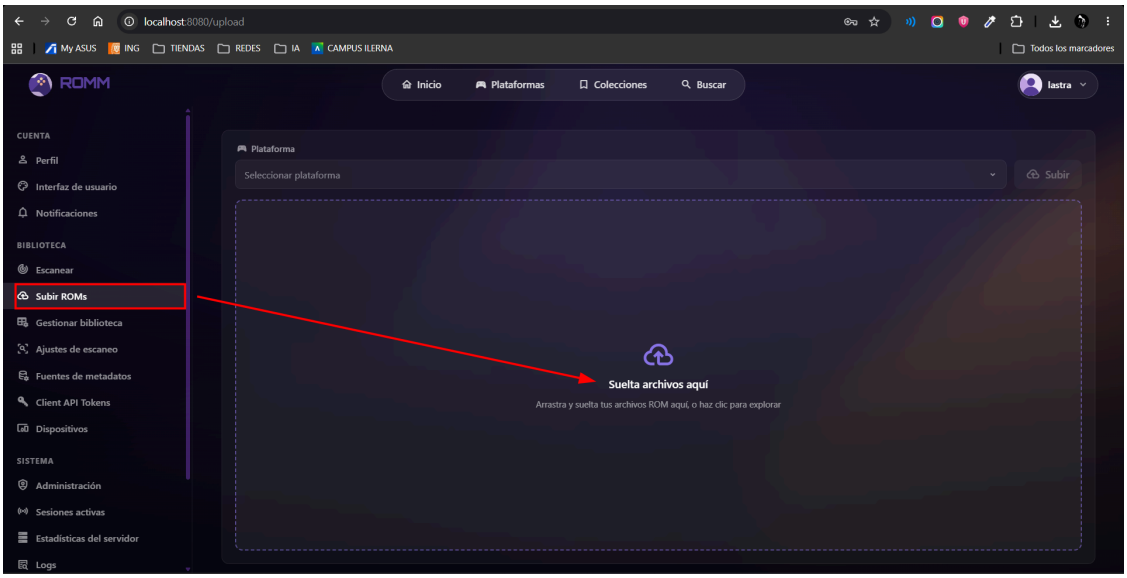


Figura 9. Interfaz de carga de RomM (localhost:8080/upload) con zona interactiva de suelta de archivos.

Selección de Plataforma y Cola de Carga:

Se selecciona en el desplegable superior la plataforma correspondiente: **Nintendo 64**. A continuación, se arrastra la colección de archivos desde el explorador de Windows hacia el navegador, generándose una cola de preparación con más de 1.100 títulos listos para ingresar en el servidor. Finalmente, se acciona el botón **Subir**.

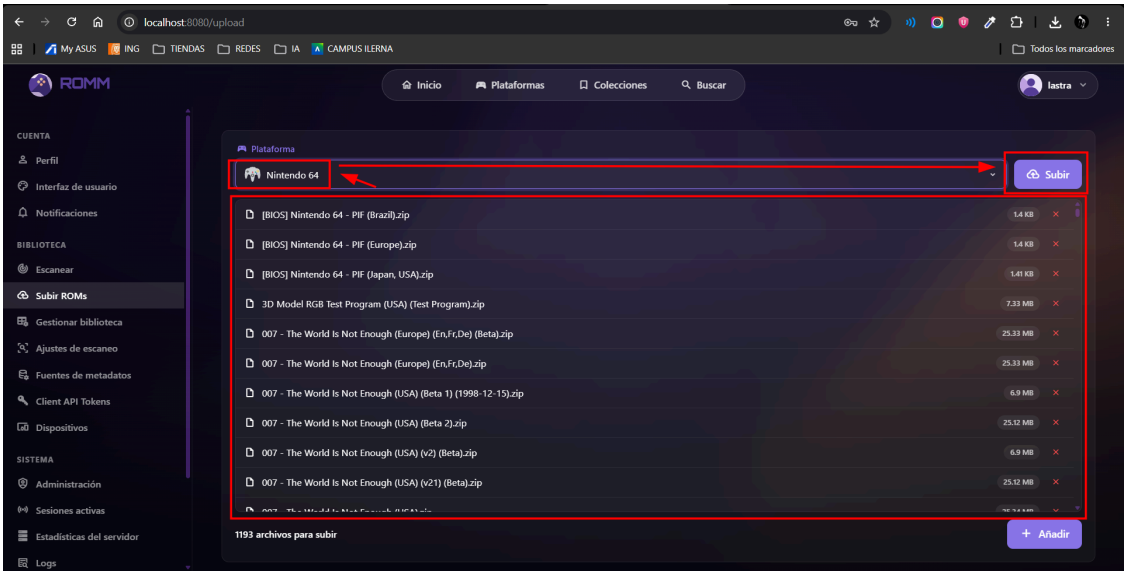


Figura 10. Cola de 1.193 archivos asignados a Nintendo 64 preparados para su transferencia masiva.

9. Fase 5 (Cont.): Monitorización de la Subida por Lotes

La transferencia de colecciones voluminosas requiere un sistema robusto de procesamiento concurrente que garantice que ningún archivo resulte truncado o falto de validación en el servidor.

Procesamiento Concurrente y Barras de Estado:

Al iniciar la carga, RomM despliega una ventana flotante modal en la esquina inferior derecha que monitoriza en tiempo real el progreso de cada archivo individualmente (*Subiendo 981 archivos...*):

- **Barras de Progreso Dinámicas:** Reflejan el caudal de subida en tiempo real y el porcentaje transferido para cada título específico.
- **Validación Inmediata:** Al completarse la transmisión de un archivo, el sistema realiza el cálculo de hash, verifica la extensión y muestra un distintivo verde de conformidad.
- **Almacenamiento en Volúmenes:** Los archivos se guardan directamente en el volumen de almacenamiento persistente montado en `/romm/library/n64/`, quedando listos para el motor de escaneado y etiquetado automático.

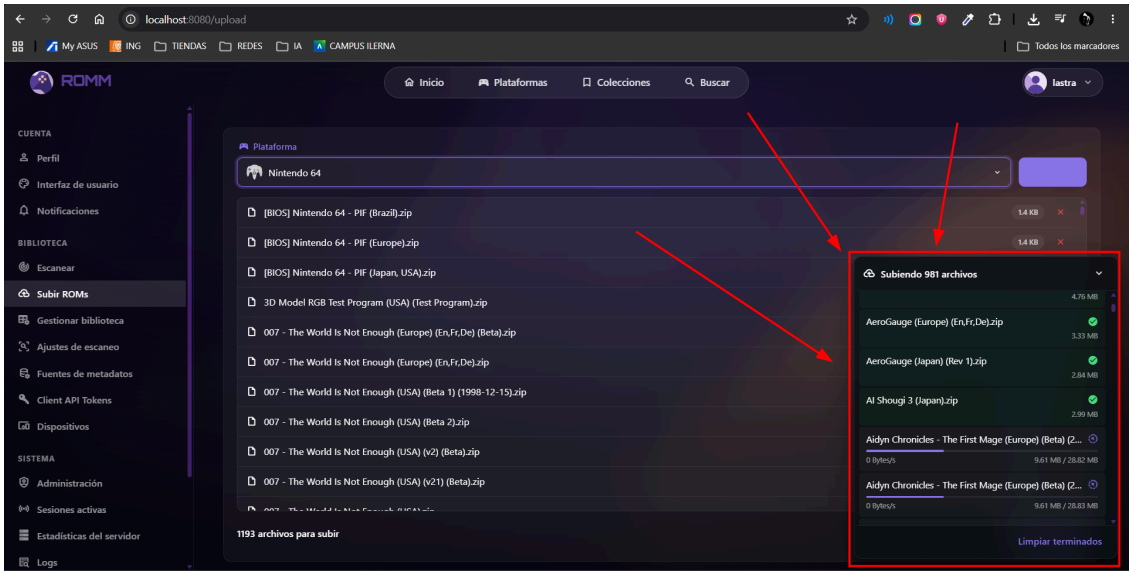


Figura 11. Monitorización en tiempo real del lote de subida con estados completados y barras de progreso.

10. Fase 6: Catálogo de la Biblioteca y Emulación Web

Una vez procesados los archivos, RomM estructura automáticamente la colección en su catálogo visual, asociando carátulas, banderas de región y herramientas de ejecución directa.

Exploración de la Colección en RomM:

Al ingresar en el menú superior **Plataformas** y seleccionar *Nintendo 64* (`http://localhost:8080/platform/1`), los títulos se presentan ordenados alfabéticamente en un elegante formato de cuadrícula. Cada tarjeta incluye la portada, el título completo y las insignias de región geográfica (USA, Japón, Europa).

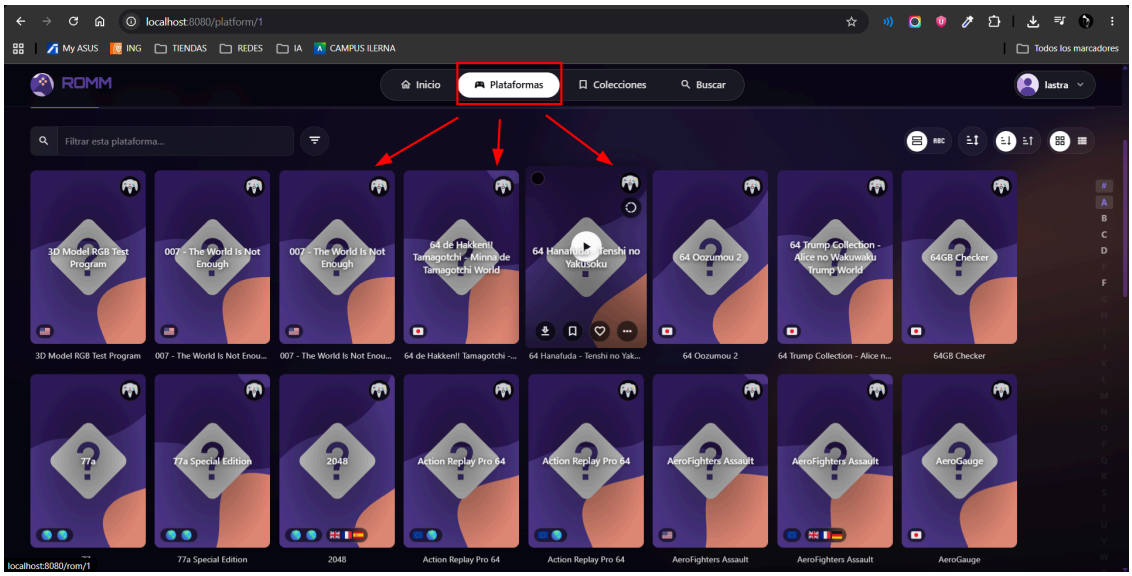


Figura 12. Catálogo interactivo de la colección de Nintendo 64 con metadatos y banderas de región.

Emulación Integrada en el Navegador Web (EmulatorJS):

Al hacer clic sobre cualquiera de los títulos (en este ejemplo, *Army Men: Sarge's Heroes* en `localhost:8080/rom/18/ejs`), RomM inicializa el motor de emulación web embebido **EmulatorJS**. El juego se ejecuta fluidamente a través de WebAssembly y WebGL dentro de la propia ventana del navegador, sin necesidad de instalar clientes emuladores adicionales ni configurar rutas complejas de plugins en el equipo cliente.



Figura 13. Ejecución directa del juego en el navegador web mediante el núcleo EmulatorJS de RomM.

11. Fase 7: Mantenimiento y Buenas Prácticas

Para asegurar la estabilidad, seguridad y longevidad de la biblioteca de videojuegos autoalojada, es conveniente adoptar una serie de pautas de administración regular.

Copias de Seguridad de la Base de Datos y Biblioteca:

- **Exportación de MariaDB:** Realizar volcados periódicos de la base de datos relacional para no perder el histórico de metadatos, tags y tiempos de juego:

  > **Comando de Backup MariaDB**
  > `docker compose exec romm-db mysqldump -u romm -p romm > backup_romm.sql`

- **Sincronización del Directorio de ROMs:** Mantener una copia espejo del directorio montado (`/romm/library`) en un disco duro externo o servicio de almacenamiento secundario.

Actualización del Stack de Contenedores:

Para recibir las últimas mejoras de rendimiento y parches de seguridad desarrollados por el equipo oficial de RomM, se recomienda actualizar periódicamente las imágenes Docker:

> **Actualización con Docker Compose**
> `docker compose pull && docker compose up -d`

Conclusión del Proyecto:

La combinación de **Docker Compose**, **RomM**, **qBittorrent** y los repositorios de preservación de **Minerva Archive (No-Intro)** ofrece una solución profesional, robusta y completamente autocontenida para la gestión integral de videojuegos clásicos. Este flujo de trabajo garantiza la preservación fiel del software frente a la obsolescencia del hardware original, facilitando el acceso inmediato y multiplataforma mediante emulación web segura y respetando rigurosamente los principios legales de propiedad intelectual.